

Perfect Secrecy, the One-Time Pad, and Computational Security

CS 171: Cryptography

Sanjam Garg

September 3, 2026

Overview

- 1 Perfect Secrecy
- 2 The One-Time Pad
- 3 Computational Security
- 4 Polynomial and Negligible Functions
- 5 Computationally Secure Encryption
- 6 Summary

Defining Secure Encryption: Formally

Definition 1 (Perfect Secrecy)

An encryption scheme $(\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is **perfectly secret** if for every probability distribution over $\mathcal{M}$, every message $m \in \mathcal{M}$, and every ciphertext c with $\Pr[C = c] > 0$:

$$\Pr[M = m \mid C = c] = \Pr[M = m].$$

Defining Secure Encryption: Formally

Definition 1 (Perfect Secrecy)

An encryption scheme $(\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is **perfectly secret** if for every probability distribution over $\mathcal{M}$, every message $m \in \mathcal{M}$, and every ciphertext c with $\Pr[C = c] > 0$:

$$\Pr[M = m \mid C = c] = \Pr[M = m].$$

Equivalent formulation

Or, equivalently, if for every two messages $m, m' \in \mathcal{M}$ and every ciphertext c (in the ciphertext space):

$$\Pr[\text{Enc}_K(m) = c] = \Pr[\text{Enc}_K(m') = c].$$

Definition 3 (Game Style)

Game $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}$

- 1 $\mathcal{A}$ outputs $m_0, m_1 \in \mathcal{M}$.
- 2 $b \leftarrow \{0, 1\}$, $k \leftarrow \text{Gen}()$, $c \leftarrow \text{Enc}_k(m_b)$.
- 3 Challenge ciphertext c is given to $\mathcal{A}$.
- 4 $\mathcal{A}$ outputs b' .
- 5 Output 1 if $b = b'$, and 0 otherwise.

Definition 3 (Game Style)

Game $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}$

- 1 $\mathcal{A}$ outputs $m_0, m_1 \in \mathcal{M}$.
- 2 $b \leftarrow \{0, 1\}$, $k \leftarrow \text{Gen}()$, $c \leftarrow \text{Enc}_k(m_b)$.
- 3 Challenge ciphertext c is given to $\mathcal{A}$.
- 4 $\mathcal{A}$ outputs b' .
- 5 Output 1 if $b = b'$, and 0 otherwise.

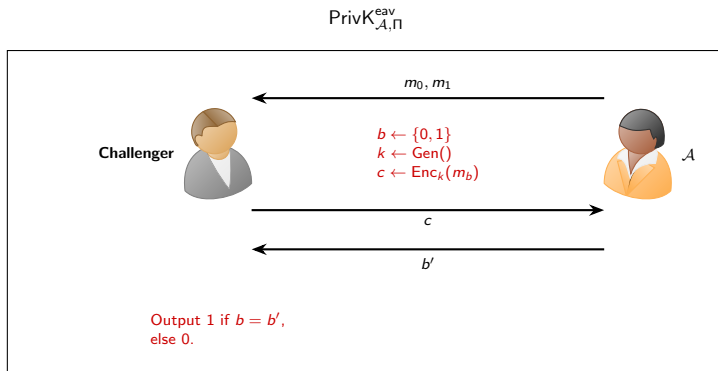
Definition 2 (Perfect Indistinguishability)

Encryption scheme $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is **perfectly indistinguishable** if $\forall \mathcal{A}$ it holds that

$$\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}} = 1] = \frac{1}{2}.$$

- *eav* is for *eavesdropper*.

$\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}$: Challenger vs. Adversary



Remark

$\mathcal{A}$ can always succeed with probability $\frac{1}{2}$ by guessing b' at random; security requires it can do no better.

Two Equivalent Definitions

Lemma (prove on your own)

An encryption scheme is **perfectly secret** if and only if it is **perfectly indistinguishable**.

- The two definitions capture the same intuition: the ciphertext reveals nothing about which message was encrypted.
- The game-based (indistinguishability) formulation will be the template for all later security definitions.

The One-Time Pad

Definition 3 (One-Time Pad)

Fix an integer ℓ , then let $\mathcal{M} = \mathcal{K} = \mathcal{C} = \{0, 1\}^\ell$:

- Gen: output a uniform value from $\mathcal{K}$.
- $\text{Enc}_k(m)$: where $m \in \{0, 1\}^\ell$, output $c := k \oplus m$.
- $\text{Dec}_k(c)$: output $m := k \oplus c$.

The One-Time Pad

Definition 3 (One-Time Pad)

Fix an integer ℓ , then let $\mathcal{M} = \mathcal{K} = \mathcal{C} = \{0, 1\}^\ell$:

- Gen: output a uniform value from $\mathcal{K}$.
- $\text{Enc}_k(m)$: where $m \in \{0, 1\}^\ell$, output $c := k \oplus m$.
- $\text{Dec}_k(c)$: output $m := k \oplus c$.

Correctness

$$\text{Dec}_k(\text{Enc}_k(m)) = k \oplus k \oplus m = m.$$

The One-Time Pad

Definition 3 (One-Time Pad)

Fix an integer ℓ , then let $\mathcal{M} = \mathcal{K} = \mathcal{C} = \{0, 1\}^\ell$:

- Gen: output a uniform value from $\mathcal{K}$.
- $\text{Enc}_k(m)$: where $m \in \{0, 1\}^\ell$, output $c := k \oplus m$.
- $\text{Dec}_k(c)$: output $m := k \oplus c$.

Correctness

$$\text{Dec}_k(\text{Enc}_k(m)) = k \oplus k \oplus m = m.$$

Security (perfect secrecy)

$$\begin{aligned} \forall m, c : \Pr[\text{Enc}_K(m) = c] &= 2^{-\ell}. \quad \text{Or,} \\ \forall m, m', c : \Pr[\text{Enc}_K(m) = c] &= \Pr[\text{Enc}_K(m') = c]. \end{aligned}$$

One-Time Pad: Good and Bad

- One-Time Pad achieves **perfect security**.
 - Has been used in the past.

One-Time Pad: Good and Bad

- One-Time Pad achieves **perfect security**.
 - Has been used in the past.
- Not used anymore, why not?

One-Time Pad: Good and Bad

- One-Time Pad achieves **perfect security**.
 - Has been used in the past.
- Not used anymore, why not?
 - The key is **as long as the message**.

One-Time Pad: Good and Bad

- One-Time Pad achieves **perfect security**.
 - Has been used in the past.
- Not used anymore, why not?
 - The key is **as long as the message**.
 - Can't reuse the key.

One-Time Pad: Good and Bad

- One-Time Pad achieves **perfect security**.
 - Has been used in the past.
- Not used anymore, why not?
 - The key is **as long as the message**.
 - Can't reuse the key.
 - Broken under a **known-plaintext attack**.

Key reuse is fatal

If $c_1 = k \oplus m_1$ and $c_2 = k \oplus m_2$, then $c_1 \oplus c_2 = m_1 \oplus m_2$, leaking information about the messages.

Can We Make $|\mathcal{M}| > |\mathcal{K}|$?

The question

The one-time pad needs a key as long as the message ($|\mathcal{K}| = |\mathcal{M}|$). Can a *perfectly secret* scheme use a key space smaller than its message space, i.e. $|\mathcal{M}| > |\mathcal{K}|$?

- Intuitively: more messages than keys means some ciphertext cannot “cover” every message, which should break perfect secrecy.
- The next theorem shows the answer is **no**.

Optimality of the One-Time Pad

Theorem 4 (Shannon)

If $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is a perfectly secret encryption scheme with message space $\mathcal{M}$ and key space $\mathcal{K}$, then $|\mathcal{M}| \leq |\mathcal{K}|$.

Optimality of the One-Time Pad

Theorem 4 (Shannon)

If $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is a perfectly secret encryption scheme with message space $\mathcal{M}$ and key space $\mathcal{K}$, then $|\mathcal{M}| \leq |\mathcal{K}|$.

Proof idea

Assume $|\mathcal{K}| < |\mathcal{M}|$; we show Π cannot be perfectly secret.

Optimality of the One-Time Pad

Theorem 4 (Shannon)

If $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is a perfectly secret encryption scheme with message space $\mathcal{M}$ and key space $\mathcal{K}$, then $|\mathcal{M}| \leq |\mathcal{K}|$.

Proof idea

Assume $|\mathcal{K}| < |\mathcal{M}|$; we show Π cannot be perfectly secret.

- Fix a ciphertext c and let $\mathcal{M}(c) = \{ m \mid m = \text{Dec}_k(c) \text{ for some } k \in \mathcal{K} \}$.

Optimality of the One-Time Pad

Theorem 4 (Shannon)

If $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is a perfectly secret encryption scheme with message space $\mathcal{M}$ and key space $\mathcal{K}$, then $|\mathcal{M}| \leq |\mathcal{K}|$.

Proof idea

Assume $|\mathcal{K}| < |\mathcal{M}|$; we show Π cannot be perfectly secret.

- Fix a ciphertext c and let $\mathcal{M}(c) = \{ m \mid m = \text{Dec}_k(c) \text{ for some } k \in \mathcal{K} \}$.
- Then $|\mathcal{M}(c)| \leq |\mathcal{K}| < |\mathcal{M}|$, so $\exists m' \in \mathcal{M}$ with $m' \notin \mathcal{M}(c)$.

Optimality of the One-Time Pad

Theorem 4 (Shannon)

If $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is a perfectly secret encryption scheme with message space $\mathcal{M}$ and key space $\mathcal{K}$, then $|\mathcal{M}| \leq |\mathcal{K}|$.

Proof idea

Assume $|\mathcal{K}| < |\mathcal{M}|$; we show Π cannot be perfectly secret.

- Fix a ciphertext c and let $\mathcal{M}(c) = \{ m \mid m = \text{Dec}_k(c) \text{ for some } k \in \mathcal{K} \}$.
- Then $|\mathcal{M}(c)| \leq |\mathcal{K}| < |\mathcal{M}|$, so $\exists m' \in \mathcal{M}$ with $m' \notin \mathcal{M}(c)$.
- For such m' : $\Pr[M = m' \mid C = c] = 0 \neq \Pr[M = m']$, contradicting perfect secrecy.



Computational Security

- A **relaxation** of perfect security:
 - Security only against **efficient** adversaries.
 - Security can fail with some **very small** probability.
- Two approaches:
 - **Concrete** security.
 - **Asymptotic** security.

Why relax?

This relaxation is what allows schemes with short keys (e.g. keys much shorter than the messages) to be considered secure — escaping Shannon's bound.

Definition 5 (Concrete Security)

A scheme is (t, ϵ) -**secure** if any adversary running for time at most t succeeds in breaking the scheme with probability at most ϵ .

Definition 5 (Concrete Security)

A scheme is (t, ϵ) -**secure** if any adversary running for time at most t succeeds in breaking the scheme with probability at most ϵ .

Example 6

Consider an encryption scheme that is $(2^{128}, 2^{-60})$ -secure.

Definition 5 (Concrete Security)

A scheme is (t, ϵ) -**secure** if any adversary running for time at most t succeeds in breaking the scheme with probability at most ϵ .

Example 6

Consider an encryption scheme that is $(2^{128}, 2^{-60})$ -secure.

- 2^{80} is roughly the computation that can be performed by super-computers in one year or so.

Definition 5 (Concrete Security)

A scheme is (t, ϵ) -**secure** if any adversary running for time at most t succeeds in breaking the scheme with probability at most ϵ .

Example 6

Consider an encryption scheme that is $(2^{128}, 2^{-60})$ -secure.

- 2^{80} is roughly the computation that can be performed by super-computers in one year or so.
- 2^{-60} is roughly the probability that an event happens once every 100 billion years.

What's Wrong with Concrete Security?

- Concrete security is **essential** in choosing scheme parameters in practice.

What's Wrong with Concrete Security?

- Concrete security is **essential** in choosing scheme parameters in practice.
- However, it doesn't yield a clean theory:
 - Depends on the computational model.
 - Need to change schemes as (t, ϵ) need to be updated.

What's Wrong with Concrete Security?

- Concrete security is **essential** in choosing scheme parameters in practice.
- However, it doesn't yield a clean theory:
 - Depends on the computational model.
 - Need to change schemes as (t, ϵ) need to be updated.
- Need schemes that allow **tuning** (t, ϵ) as desired.

Asymptotic Security

- Introduce a **security parameter** n (known to the adversary).

Asymptotic Security

- Introduce a **security parameter** n (known to the adversary).
- All honest parties run in **polynomial time** in n .

Asymptotic Security

- Introduce a **security parameter** n (known to the adversary).
- All honest parties run in **polynomial time** in n .
- Security can be tuned by changing n .

Asymptotic Security

- Introduce a **security parameter** n (known to the adversary).
- All honest parties run in **polynomial time** in n .
- Security can be tuned by changing n .
 - t and ϵ are now functions of n :

Asymptotic Security

- Introduce a **security parameter** n (known to the adversary).
- All honest parties run in **polynomial time** in n .
- Security can be tuned by changing n .
 - t and ϵ are now functions of n :
 - $t \rightarrow$ probabilistic polynomial time (**PPT**) in n .

Asymptotic Security

- Introduce a **security parameter** n (known to the adversary).
- All honest parties run in **polynomial time** in n .
- Security can be tuned by changing n .
 - t and ϵ are now functions of n :
 - $t \rightarrow$ probabilistic polynomial time (**PPT**) in n .
 - $\epsilon \rightarrow$ a **negligible** function in n .

Polynomial and Negligible

Polynomial

A function $f : \mathbb{Z}^+ \rightarrow \mathbb{Z}^+$ is **polynomial** if there exists c such that for all large enough n :

$$f(n) < n^c.$$

Polynomial and Negligible

Polynomial

A function $f : \mathbb{Z}^+ \rightarrow \mathbb{Z}^+$ is **polynomial** if there exists c such that for all large enough n :

$$f(n) < n^c.$$

Negligible

A function $f : \mathbb{Z}^+ \rightarrow [0, 1]$ is **negligible** if for every polynomial p it holds that for all large enough n :

$$f(n) < \frac{1}{p(n)}.$$

- Typical example: $f(n) = \text{poly}(n) \cdot 2^{-\alpha n}$.

Negligible Function (Formally)

Definition 7 (Negligible Function)

A function $f : \mathbb{Z}^+ \rightarrow [0, 1]$ is **negligible** if for every polynomial p ,

$$\exists N \in \mathbb{Z}^+ \quad \forall n > N : \quad f(n) < \frac{1}{p(n)}.$$

Negligible Function (Formally)

Definition 7 (Negligible Function)

A function $f : \mathbb{Z}^+ \rightarrow [0, 1]$ is **negligible** if for every polynomial p ,

$$\exists N \in \mathbb{Z}^+ \quad \forall n > N : \quad f(n) < \frac{1}{p(n)}.$$

Example 8

Prove that $f(n) = 2^{-n}$ is a negligible function.

- For any polynomial p , eventually $2^{-n} < 1/p(n)$ since 2^n grows faster than every polynomial.

Is This a Negligible Function?

- $f(n) = 2^{-\sqrt{n}}$?

Is This a Negligible Function?

- $f(n) = 2^{-\sqrt{n}}$
 - **Yes** — $2^{-\sqrt{n}}$ eventually drops below $1/p(n)$ for every polynomial p .

Is This a Negligible Function?

- $f(n) = 2^{-\sqrt{n}}$
 - **Yes** — $2^{-\sqrt{n}}$ eventually drops below $1/p(n)$ for every polynomial p .
- $f(n) = n^{-\log n}$

Is This a Negligible Function?

- $f(n) = 2^{-\sqrt{n}}$?
 - **Yes** — $2^{-\sqrt{n}}$ eventually drops below $1/p(n)$ for every polynomial p .
- $f(n) = n^{-\log n}$?
 - **Yes** — for every constant c , once $\log n > c$ we have $n^{-\log n} < n^{-c}$.

Is This a Negligible Function?

- $f(n) = 2^{-\sqrt{n}}$?
 - **Yes** — $2^{-\sqrt{n}}$ eventually drops below $1/p(n)$ for every polynomial p .
- $f(n) = n^{-\log n}$?
 - **Yes** — for every constant c , once $\log n > c$ we have $n^{-\log n} < n^{-c}$.
- $f(n) = \begin{cases} 2^{-n} & \text{for } n \bmod 2 = 0 \\ n^{-c} & \text{for } n \bmod 2 = 1 \end{cases}$ (for some constant c)?

Is This a Negligible Function?

- $f(n) = 2^{-\sqrt{n}}$
 - **Yes** — $2^{-\sqrt{n}}$ eventually drops below $1/p(n)$ for every polynomial p .
- $f(n) = n^{-\log n}$
 - **Yes** — for every constant c , once $\log n > c$ we have $n^{-\log n} < n^{-c}$.
- $f(n) = \begin{cases} 2^{-n} & \text{for } n \bmod 2 = 0 \\ n^{-c} & \text{for } n \bmod 2 = 1 \end{cases}$ (for some constant c)?
 - **No** — for every odd n , $f(n) = n^{-c}$ is *not* below $1/p(n)$ for $p(n) = n^c$; the bound must hold for *all* large enough n , not just the even ones.

Choice of Polynomial and Negligible

- Using PPT for “efficient” machines is borrowed from **complexity theory**.
- Also some nice **closure properties**:
 - $\text{poly}(n) \cdot \text{poly}(n)$ is still $\text{poly}(n)$.
 - $\text{poly}(n) \cdot \text{negl}(n)$ is still $\text{negl}(n)$.

Why these closures matter

They let us compose efficient algorithms (still efficient) and union-bound over polynomially many negligible events (still negligible) — the workhorses of security proofs.

Concrete vs. Asymptotic

Concrete

A scheme is (t, ϵ) -**secure** if any adversary running for time at most t succeeds in breaking the scheme with probability at most ϵ .

Concrete vs. Asymptotic

Concrete

A scheme is (t, ϵ) -**secure** if any adversary running for time at most t succeeds in breaking the scheme with probability at most ϵ .

Asymptotic

A scheme is **secure** if any PPT adversary succeeds in breaking the scheme with at most **negligible** probability.

fixed t, ϵ



$$t = \text{poly}(n), \quad \epsilon = \text{negl}(n)$$

Defining Computationally Secure Encryption (Syntax)

Definition 9 (Private-Key Encryption Scheme)

A **private-key encryption scheme** is a tuple of algorithms $(\text{Gen}, \text{Enc}, \text{Dec})$:

- $\text{Gen}(1^n)$: outputs a key k (assume $|k| > n$).
- $\text{Enc}_k(m)$: takes key k and message $m \in \{0, 1\}^*$ as input; outputs ciphertext $c \leftarrow \text{Enc}_k(m)$.
- $\text{Dec}_k(c)$: takes key k and ciphertext c as input; outputs $m := \text{Dec}_k(c)$ or “error”.

Defining Computationally Secure Encryption (Syntax)

Definition 9 (Private-Key Encryption Scheme)

A **private-key encryption scheme** is a tuple of algorithms $(\text{Gen}, \text{Enc}, \text{Dec})$:

- $\text{Gen}(1^n)$: outputs a key k (assume $|k| > n$).
- $\text{Enc}_k(m)$: takes key k and message $m \in \{0, 1\}^*$ as input; outputs ciphertext $c \leftarrow \text{Enc}_k(m)$.
- $\text{Dec}_k(c)$: takes key k and ciphertext c as input; outputs $m := \text{Dec}_k(c)$ or “error”.

Correctness

For all n , all k output by $\text{Gen}(1^n)$, and all $m \in \{0, 1\}^*$:

$$\text{Dec}_k(\text{Enc}_k(m)) = m.$$

Computational Indistinguishability

Game $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(n)$

- 1 $\mathcal{A}$ (given 1^n) outputs $m_0, m_1 \in \{0, 1\}^*$ with $|m_0| = |m_1|$.
- 2 $b \leftarrow \{0, 1\}$, $k \leftarrow \text{Gen}(1^n)$, $c \leftarrow \text{Enc}_k(m_b)$.
- 3 Challenge ciphertext c is given to $\mathcal{A}$; it outputs b' .
- 4 Output 1 if $b = b'$, and 0 otherwise.

Computational Indistinguishability

Game $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(n)$

- 1 $\mathcal{A}$ (given 1^n) outputs $m_0, m_1 \in \{0, 1\}^*$ with $|m_0| = |m_1|$.
- 2 $b \leftarrow \{0, 1\}$, $k \leftarrow \text{Gen}(1^n)$, $c \leftarrow \text{Enc}_k(m_b)$.
- 3 Challenge ciphertext c is given to $\mathcal{A}$; it outputs b' .
- 4 Output 1 if $b = b'$, and 0 otherwise.

Definition 10 (Computational Indistinguishability (EAV-security))

Π is **computationally indistinguishable** if for every **PPT** $\mathcal{A}$:

$$\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n).$$

Caveat

Does **not** hide message length!

Distinguishing Variant

Game $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(n, d)$

- ① $\mathcal{A}$ (given 1^n) outputs $m_0, m_1 \in \{0, 1\}^*$ with $|m_0| = |m_1|$.
 - ② $b = d$, $k \leftarrow \text{Gen}(1^n)$, $c \leftarrow \text{Enc}_k(m_b)$.
 - ③ Challenge ciphertext c is given to $\mathcal{A}$.
 - ④ $\mathcal{A}$ outputs b' . The output of $\mathcal{A}$ is $\text{out}_{\mathcal{A}}(\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(1^n, d))$.
- Here, $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(1^n, d)$ is the same as $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(1^n)$ except that we set $b = d$.

Distinguishing Variant

Game $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(n, d)$

- ① $\mathcal{A}$ (given 1^n) outputs $m_0, m_1 \in \{0, 1\}^*$ with $|m_0| = |m_1|$.
- ② $b = d$, $k \leftarrow \text{Gen}(1^n)$, $c \leftarrow \text{Enc}_k(m_b)$.
- ③ Challenge ciphertext c is given to $\mathcal{A}$.
- ④ $\mathcal{A}$ outputs b' . The output of $\mathcal{A}$ is $\text{out}_{\mathcal{A}}(\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(1^n, d))$.

- Here, $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(1^n, d)$ is the same as $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(1^n)$ except that we set $b = d$.

Definition 11 (Indistinguishability, distinguishing form)

Π is computationally indistinguishable if for every PPT $\mathcal{A}$:

$$|\Pr[\text{out}_{\mathcal{A}}(\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(1^n, 1)) = 1] - \Pr[\text{out}_{\mathcal{A}}(\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(1^n, 0)) = 1]| \leq \text{negl}(n).$$

Summary

- **Perfect secrecy** $\equiv$ **perfect indistinguishability** (game $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}$, winning prob. exactly $\frac{1}{2}$).
- **One-time pad**: $c = k \oplus m$ — perfectly secret, but key as long as message and not reusable.
- **Shannon's bound**: perfect secrecy requires $|\mathcal{K}| \geq |\mathcal{M}|$.
- **Computational security** relaxes this: efficient (PPT) adversaries, negligible failure.
 - **Concrete**: (t, ϵ) -security.
 - **Asymptotic**: security parameter n , $\text{poly}(n)$ time, $\text{negl}(n)$ advantage.
- **EAV-security**: $\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$ for all PPT $\mathcal{A}$.